



UNIVERSIDADE FEDERAL DE SANTA CATARINA
CAMPUS FLORIANÓPOLIS
CURSO DE GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

Lucas Bueno Cesario

**POLYGLOT IMPORT CSV: UMA SOLUÇÃO PARA IMPORTAÇÃO DE
DOCUMENTOS CSV PARA MÚLTIPLOS BANCOS DE DADOS**

Florianópolis
2026

Lucas Bueno Cesario

**POLYGLOT IMPORT CSV: UMA SOLUÇÃO PARA IMPORTAÇÃO DE
DOCUMENTOS CSV PARA MÚLTIPLOS BANCOS DE DADOS**

Trabalho de Conclusão de Curso submetido ao Curso de Graduação em Ciência da Computação do Centro Tecnológico da Universidade Federal de Santa Catarina para obtenção do título de Bacharel em Ciência da Computação.

Orientador: Prof. Ronaldo dos Santos Mello, Dr.

Florianópolis
2026

Ficha catalográfica para trabalhos acadêmicos.

Cesario, Lucas Bueno

Polyglot Import CSV : Uma solução para importação de documentos CSV para múltiplos bancos de dados / Lucas Bueno Cesario ; orientador, Ronaldo dos Santos Mello, 2026.

49 p.

Trabalho de Conclusão de Curso (graduação) - Universidade Federal de Santa Catarina, Centro Tecnológico, Graduação em Ciências da Computação, Florianópolis, 2026.

Inclui referências.

1. Ciências da Computação. 2. Persistência poliglota. 3. Importação de dados CSV. 4. NoSQL. 5. Software Livre. I. Mello, Ronaldo dos Santos. II. Universidade Federal de Santa Catarina. Graduação em Ciências da Computação. III. Título.

Lucas Bueno Cesario

**POLYGLOT IMPORT CSV: UMA SOLUÇÃO PARA IMPORTAÇÃO DE
DOCUMENTOS CSV PARA MÚLTIPLOS BANCOS DE DADOS**

Este Trabalho de Conclusão de Curso foi julgado adequado para obtenção do Título de “Bacharel em Ciência da Computação” e aprovado em sua forma final pelo Curso de Graduação em Ciências da Computação.

Florianópolis, _____ de _____ de 2026.

Profa. Lúcia Helena Martins Pacheco, Dra.
Coordenadora do Curso

Banca Examinadora:

Prof. Ronaldo dos Santos Mello, Dr.
Orientador
Universidade Federal de Santa Catarina

Profa. Carina Friedrich Dorneles, Dra.
Avaliadora
Universidade Federal de Santa Catarina

Prof. Jônata Tyska Carvalho, Dr.
Avaliador
Universidade Federal de Santa Catarina

RESUMO

A persistência poliglota é essencial na arquitetura contemporânea de bancos de dados, integrando diversas tecnologias de armazenamento em uma mesma aplicação. A ferramenta *Polyglot Import CSV* destaca-se como solução para otimizar a importação de dados CSV em diversos bancos de dados simultaneamente, superando desafios da persistência poliglota. Optou-se pelo formato CSV por ser amplamente utilizado na importação de dados pelos diversos SGBDs e por ser um formato simples e familiar tanto a humanos quanto a sistemas. Por meio de configurações declarativas em arquivos JSON, a ferramenta oferece suporte ao principal representante de código aberto dos bancos de dados relacionais — o PostgreSQL — e aos principais representantes dos bancos de dados NoSQL: Redis (chave-valor), MongoDB (documento), Apache Cassandra (colunar) e Neo4j (grafo). Não foram identificadas ferramentas equivalentes que realizem, a partir de um único arquivo CSV e de regras declarativas, a importação simultânea para múltiplos paradigmas de dados, o que evidencia a originalidade da proposta. A publicação do código aberto da ferramenta impulsiona avanços colaborativos na dinâmica arquitetura moderna de bancos de dados.

Palavras-chave: Persistência poliglota. Ferramenta *Polyglot Import CSV*. Importação de dados CSV. JSON. SQL. NoSQL. Software Livre.

ABSTRACT

Polyglot persistence is essential in contemporary database architecture, integrating various storage technologies within the same application. The Polyglot Import CSV tool stands out as a solution for optimizing CSV data import into multiple databases simultaneously, overcoming polyglot persistence challenges. The CSV format was chosen because it is widely used for data import by various DBMSs and because it is a simple and familiar format for both humans and systems. Through declarative configurations in JSON files, the tool supports the leading open-source representative of relational databases — PostgreSQL — and the main representatives of NoSQL databases: Redis (key-value), MongoDB (document), Apache Cassandra (columnar), and Neo4j (graph). No equivalent tools were identified that perform, from a single CSV file and declarative rules, the simultaneous import to multiple data paradigms, which highlights the originality of the proposal. The open-source publication of the tool drives collaborative advances in the dynamic modern database architecture.

Keywords: *Polyglot persistence. Polyglot Import CSV tool. CSV data import. JSON. SQL. NoSQL. Free Software.*

LISTA DE ILUSTRAÇÕES

Figura 1 – Exemplo de aplicação da Persistência Poliglota em um e-commerce.	15
Figura 2 – Diferentes tipos de modelos de dados NoSQL.	16
Figura 3 – Modelo chave-valor: chaves apontando para valores arbitrários (Redis).	17
Figura 4 – Modelo colunar: chaves de linha e famílias de colunas (Cassandra).	18
Figura 5 – Modelo de documento: documentos autocontidos com campos variáveis (MongoDB).	19
Figura 6 – Modelo de grafo: nós e arestas tipadas com propriedades (Neo4j). .	20
Figura 7 – Estrutura dos dois JSON Schemas de configuração (importação e conexão).	29
Figura 8 – Fluxo de carregamento e validação dos dois arquivos de configuração.	37
Figura 9 – Algoritmo de execução da importação Polyglot Import CSV.	40

LISTA DE TABELAS

Tabela 1 – Campos de mapeamento de coluna (<code>columnSpec</code>).	31
Tabela 2 – Campos removidos na reforma de simplificação do schema.	32
Tabela 3 – Materialização de uma linha <code>action=stock</code> nos diferentes backends.	41
Tabela 4 – Contagens reportadas pelo modo <code>--dry-run</code> no cenário e-commerce.	42

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
API	<i>Application Programming Interface</i> (Interface de Programação de Aplicações)
BSON	<i>Binary JSON</i>
CLI	<i>Command Line Interface</i> (Interface de Linha de Comando)
CSV	<i>Comma-Separated Values</i> (Valores Separados por Vírgula)
DDL	<i>Data Definition Language</i> (Linguagem de Definição de Dados)
JSON	<i>JavaScript Object Notation</i>
NoSQL	<i>Not Only SQL</i>
SGBD	Sistema de Gerenciamento de Banco de Dados
SGPD	Sistema de Gerenciamento Poliglota de Dados
SQL	<i>Structured Query Language</i> (Linguagem de Consulta Estruturada)
TCC	Trabalho de Conclusão de Curso
UFSC	Universidade Federal de Santa Catarina
XML	<i>Extensible Markup Language</i>

SUMÁRIO

1	INTRODUÇÃO	11
1.1	Visão Geral	11
1.2	Objetivos	12
1.2.1	Objetivo Geral	12
1.2.2	Objetivos Específicos	12
1.3	Metodologia	12
1.4	Estrutura do Trabalho	13
2	FUNDAMENTAÇÃO TEÓRICA	14
2.1	Persistência Poliglota	14
2.2	Bancos de Dados NoSQL	16
2.2.1	Chave-valor	16
2.2.2	Colunar	17
2.2.3	Documento	18
2.2.4	Grafo	19
2.3	Bancos de Dados Multimodelo e Arquiteturas de Persistência Poliglota	20
3	TRABALHOS RELACIONADOS	22
3.1	Importação de Arquivo CSV para um Único SGBD	22
3.1.1	PostgreSQL	22
3.1.2	Redis	23
3.1.3	MongoDB	24
3.1.4	Cassandra	24
3.1.5	Neo4j	25
3.2	Importação de Arquivo CSV para SGBD Multimodelo	25
3.3	Considerações sobre os Trabalhos Relacionados	26
4	A PROPOSTA DA FERRAMENTA POLYGLOTIMPORTCSV	28
4.1	Visão Geral	28
4.2	Formato de Configuração (JSON Schema)	28
4.2.1	Visão Geral da Estrutura	29
4.2.2	Raiz, Elementos Comuns e Reforma de Simplificação	30
4.2.3	PostgreSQL	32
4.2.4	MongoDB	33
4.2.5	Apache Cassandra	34

4.2.6	Redis	34
4.2.7	Neo4j	35
4.2.8	Validação, Consistência entre Arquivos e Fluxo de Carregamento . .	36
4.3	Algoritmo de Execução da Importação	37
4.3.1	Fases do Algoritmo	37
4.3.2	Pseudocódigo	38
4.3.3	Diagrama do Fluxo	39
4.3.4	Exemplo: Uma Linha action=stock no E-commerce	40
4.4	Validação e Execução	41
4.4.1	Evidência de Execução (Cenário E-commerce)	41
5	ATIVIDADES FUTURAS	43
6	CONSIDERAÇÕES FINAIS	44
	REFERÊNCIAS	45
	APÊNDICES	47
	APÊNDICE A – JSON SCHEMAS DE CONFIGURAÇÃO NA	
	FORMA SERIALIZADA	48
A.1	Schema de Importação (import_config.schema.json)	48
A.2	Schema de Conexão (sgbd_config.schema.json)	51

1 INTRODUÇÃO

1.1 VISÃO GERAL

Nas esferas industrial e acadêmica de bancos de dados, a concepção de que um único modelo de dados não atende a todas as necessidades tem conquistado aceitação, apontando para a perspectiva de persistência poliglota no futuro. Embora os sistemas de banco de dados relacionais devam manter sua predominância, espera-se uma convivência com diversas formas de sistemas, como os NoSQL ([SADALAGE; FOWLER, 2013](#)).

Algumas pesquisas têm sido feitas na área de persistência poliglota, como sugestões de modelos de dados unificados ([CHILLÓN et al., 2023](#)), comparação de desempenho de bancos de dados multi-modelos contra abordagens de persistência poliglota ([YE et al., 2023](#)), revisões sistemáticas sobre modelagem poliglota de dados ([SILVA; BORNIA; MELLO, 2024](#)), tutoriais sobre o estado da arte e desafios abertos ([KIEHN et al., 2022](#)), e a adoção da persistência poliglota em Big Data ([KHINE; WANG, 2019](#)).

Dada essa necessidade crescente da adoção de persistência poliglota pelas aplicações, percebe-se a falta de uma ferramenta única de importação simultânea de dados para diversos SGBDs que tratem modelos de dados diferentes. Até onde se investigou (Capítulo 3), não foram identificadas ferramentas equivalentes: os importadores nativos atendem a um único SGBD por vez, e as soluções de migração ou de modelagem poliglota não realizam a materialização simultânea de um mesmo CSV em múltiplos paradigmas de dados — o que ressalta a originalidade desta proposta.

Assim sendo, a intenção deste trabalho é desenvolver a ferramenta *Polyglot Import CSV*, que importa dados armazenados em CSV para SGBDs relacionais e NoSQL. A configuração da ferramenta será baseada em dois arquivos JSON: um de importação, mapeando entidades e relacionamentos em cada SGBD, considerado seu modelo de dados; outro de configuração de conexão dos SGBDs envolvidos. A escolha pelo formato JSON deve-se à sua simplicidade e ampla adoção; sua finalidade é detalhar, de forma declarativa, o mapeamento das colunas do CSV para os modelos de dados de cada SGBD, permitindo a validação prévia da importação.

CSV (*Comma-Separated Values*) é um arquivo de valores separados por vírgulas, frequentemente visualizado em Excel e outras planilhas eletrônicas. Pode haver outros tipos de valores como delimitadores, mas o mais comum é a vírgula. Muitos sistemas e processos hoje convertem seus dados para o formato CSV para saídas de

arquivo para outros sistemas, relatórios amigáveis para humanos, disponibilização de dados públicos, e outras necessidades. É um formato de arquivo padrão com o qual humanos e sistemas já estão familiarizados ao usar e manipular.

1.2 OBJETIVOS

1.2.1 Objetivo Geral

Desenvolver a ferramenta *Polyglot Import CSV* que possibilitará a importação de dados em arquivos CSV para múltiplos SGBDs: PostgreSQL, Redis, MongoDB, Cassandra ou Neo4j.

1.2.2 Objetivos Específicos

- Projetar os arquivos de configuração JSON para que eles sigam sintaxes específicas, e flexíveis para lidar com vários modelos de dados, além de adicionar opções como filtros, escolha de chave primária e escolha do nome da coluna no banco de dados;
- Não permitir configurações incorretas, ou seja, se houver erro em algum lugar dos arquivos de configuração, a importação não deve começar até que sejam corrigidos (validação prévia da configuração);
- Avaliar o desempenho da ferramenta para diferentes volumes e modelos de dados destino;
- Disponibilizar a ferramenta de forma acessível ao usuário: inicialmente por meio de uma interface de linha de comando (CLI), com possibilidade de evolução para uma interface gráfica em trabalhos futuros.

1.3 METODOLOGIA

O desenvolvimento deste trabalho segue uma metodologia de pesquisa aplicada, adotando as seguintes etapas:

1. Estudo sobre persistência poliglota e seu estado da arte;
2. Estudo sobre os paradigmas não-relacionais que serão abrangidos pela solução: chave-valor, documento, grafo e colunar;
3. Estudo de decisão de linguagem de implementação que demonstra melhor produtividade para codificar a solução;
4. Projeto dos arquivos de configuração JSON;

5. Projeto da instrução principal de importação que utiliza os arquivos de configuração;
6. Implementação da solução “Polyglot Import CSV”;
7. Testes da solução “Polyglot Import CSV” implementada.

Na primeira etapa, foi estudado o que é a persistência poliglota, e a sua relevância no cenário atual.

Na segunda etapa, foram estudados os diversos modelos de dados não-relacionais: como cada modelo representa suas entidades; se há relacionamentos entre entidades no modelo; se é possível a existência de entidades aninhadas e multivaloradas; as limitações de cada modelo.

Na terceira etapa, foram estudadas as formas de implementar a solução usando as linguagens Java e Python e, pela simplicidade e produtividade, a linguagem Python foi escolhida.

Na quarta e quinta etapas, foram projetados os dois arquivos de configuração JSON — um para o mapeamento da importação (entidades, relacionamentos e colunas) e outro para a conexão dos SGBDs — bem como a instrução principal de importação que os utiliza.

As últimas etapas, que estão em andamento, tratam da implementação da solução proposta, “Polyglot Import CSV”, e dos testes da mesma com diferentes volumes de dados CSV e importações simples e complexas para diferentes modelos de dados destino.

1.4 ESTRUTURA DO TRABALHO

Este trabalho está estruturado em cinco capítulos principais. O primeiro aborda os objetivos, a metodologia e o contexto que levaram à elaboração deste TCC. O capítulo 2 revisa fundamentos teóricos. O capítulo 3 apresenta trabalhos relacionados. O capítulo 4 descreve a proposta da ferramenta *Polyglot Import CSV* e o capítulo 5 as atividades futuras (TCC II).

2 FUNDAMENTAÇÃO TEÓRICA

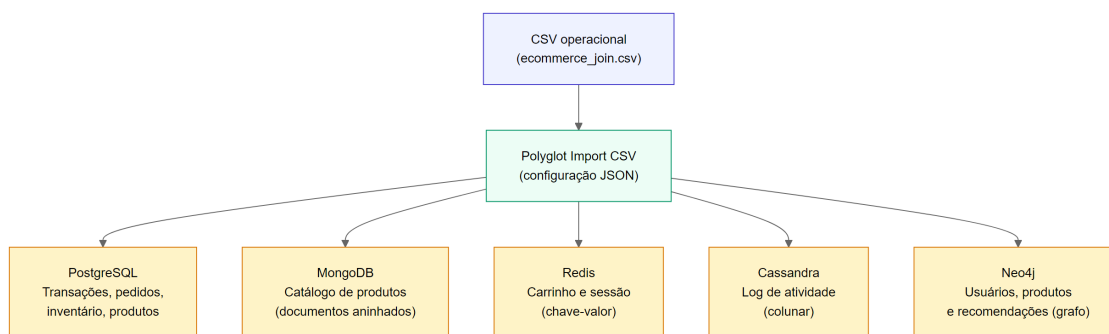
2.1 PERSISTÊNCIA POLIGLOTA

Ford (2008) introduziu o conceito de “programação poliglota” para expressar a ideia de que aplicativos devem ser desenvolvidos utilizando uma combinação de linguagens, aproveitando o fato de que diferentes linguagens são mais adequadas para lidar com diferentes problemas. Uma das consequências notáveis da programação poliglota é a transição para a “persistência poliglota”, usando a mesma analogia: pode-se dizer que múltiplas tecnologias de bancos de dados podem ser utilizadas como armazenamento persistente para diferentes tipos de casos de uso. Embora ainda exista uma quantidade considerável de dados gerenciados por sistemas relacionais, a abordagem predominante passa a ser a reflexão sobre como se deseja manipular os dados antes de determinar qual tecnologia é a mais adequada para essa manipulação (FOWLER, 2011).

A persistência poliglota abrange uma variedade de formatos de dados, incluindo estruturados, semi-estruturados e não estruturados. Dados estruturados geralmente englobam formatos como relacional, objeto-relacional, e grafos de propriedades. Dados semi-estruturados incluem principalmente documentos JSON e XML. Por fim, dados não estruturados são representados por arquivos de texto, imagens e áudios, dentre outros (YE et al., 2023).

Um domínio em que a persistência poliglota pode ser aplicada é o de um e-commerce. Nesse cenário, diferentes categorias de dados possuem características e padrões de acesso distintos — volume de escrita, frequência de leitura, necessidade de consistência, estrutura dos registros — o que torna mais adequado o uso de um SGBD específico para cada uma delas. A Figura 1 ilustra esse cenário, mapeando cada categoria de dado do e-commerce ao SGBD mais adequado, conforme sugerido a seguir:

Figura 1 – Exemplo de aplicação da Persistência Poliglota em um e-commerce.



Fonte: Elaborado pelo autor (2026).

- **Dados Financeiros/Transacionais de Pagamento e Inventário de Produtos:** Pedidos, pagamentos e estoques exigem garantias transacionais fortes (ACID) para evitar inconsistências — por exemplo, debitar um pagamento sem registrar o pedido correspondente. Além disso, consultas analíticas sobre esses dados frequentemente combinam várias entidades por meio de junções. Bancos de dados relacionais como o PostgreSQL são a escolha natural para esse perfil, pois oferecem suporte nativo a transações com consistência forte, integridade referencial e uma linguagem de consulta expressiva (SQL).
- **Dados do Catálogo de Produtos:** O catálogo de um e-commerce tende a ter esquema variável: diferentes produtos possuem atributos distintos (um livro físico com ISBN e editora; um eletrodoméstico com voltagem e garantia). Além disso, o catálogo é lido com muito maior frequência do que escrito, e cada consulta normalmente recupera o produto completo de uma vez. Bancos de dados orientados a documentos como o MongoDB são otimizados exatamente para esse perfil: armazenam cada produto como um documento autocontido (favorecendo leituras de agregados complexos), suportam esquemas flexíveis e oferecem índices secundários e *pipelines* de agregação para fins de consulta.
- **Dados da Sessão e do Carrinho de Compras do Usuário:** Dados de sessão e carrinho são acessados a cada interação do usuário durante a navegação, exigindo latência mínima, e são também inerentemente temporários — um carrinho abandonado não precisa persistir indefinidamente. Bancos de dados chave-valor em memória como o Redis oferecem latência de microssegundos, expiração automática de chaves e são horizontalmente escaláveis para absorver picos de tráfego, tornando-se a escolha ideal para dados de natureza efêmera e de acesso intenso.
- **Dados de Atividade do Usuário:** Eventos de navegação (*clickstream*), visualizações de produto e histórico de ações são gerados em alto volume e alta

velocidade. Consultas típicas percorrem o histórico de um usuário em um intervalo de tempo, ou agregam eventos por tipo de ação. Bancos de dados colunares como o Apache Cassandra são projetados para ingestão massiva com particionamento por chave primária composta (por exemplo, `(user_id, timestamp)`), garantindo escritas rápidas e leituras eficientes por usuário ou período — padrão ideal para séries temporais de atividade.

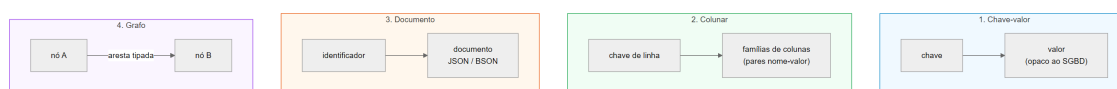
- **Dados de Recomendação:** Sistemas de recomendação dependem de relacionamentos entre entidades: usuários que compraram os mesmos produtos, produtos frequentemente adquiridos em conjunto ou usuários com perfis semelhantes. Esses padrões são naturalmente modelados como grafos, onde nós representam usuários e produtos, e arestas representam interações (comprou, avaliou, visualizou, etc.). Bancos de dados orientados a grafos como o Neo4j permitem percorrer esses relacionamentos com eficiência por meio de consultas Cypher, tarefa que em um banco de dados relacional exigiria múltiplas junções e difícil otimização.

2.2 BANCOS DE DADOS NOSQL

Os bancos de dados NoSQL podem ser divididos em subcategorias com base em seus modelos de dados. Este trabalho utiliza a classificação de [Hecht e Jablonski \(2011\)](#), que categoriza os bancos de dados NoSQL em quatro tipos: chave-valor, colunares, documentos e grafos.

A Figura 2 ilustra esses modelos de dados, que são detalhados a seguir.

Figura 2 – Diferentes tipos de modelos de dados NoSQL.



Fonte: Elaborado pelo autor (2026).

2.2.1 Chave-valor

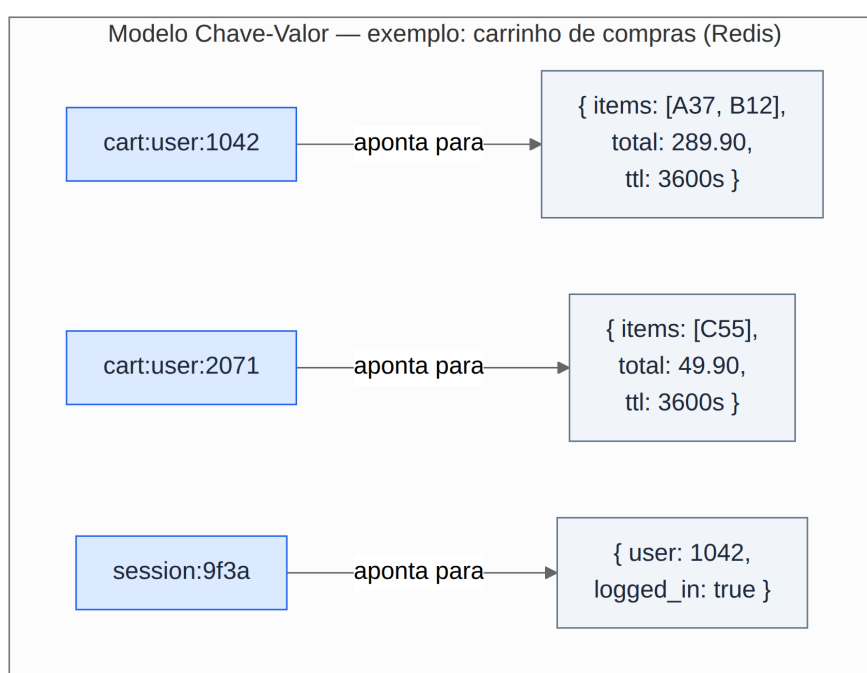
Os bancos de dados de chave-valor têm um modelo de dados simples baseado justamente em pares chave-valor, semelhante a um mapa associativo ou dicionário. A chave identifica exclusivamente o valor e é usada para armazenar e recuperar o valor no banco de dados, funcionando como uma chave primária. O valor pode ser usado para armazenar qualquer dado arbitrário, incluindo um número inteiro, uma *string*, um *array* ou um objeto, proporcionando uma representação de dados sem esquema conhecido.

Além disso, os bancos de dados de chave-valor são muito eficientes no armazenamento de dados distribuídos, mas não são adequados para cenários que requerem a definição de relacionamentos entre dados ou esquemas fortemente estruturados.

Qualquer funcionalidade que necessite desses requisitos deve ser implementada na aplicação cliente que interage com o banco de dados chave-valor.

Como os valores são opacos para os bancos de dados, esses não podem lidar com consultas e indexações a nível de dados, podendo realizar consultas apenas por meio de chaves. Um exemplo de SGBD chave-valor é o Redis, que mantém os dados tanto na memória como no disco. A Figura 3 ilustra o modelo com um exemplo de carrinho de compras e sessão de usuário, em que cada chave aponta para um valor opaco ao SGBD.

Figura 3 – Modelo chave-valor: chaves apontando para valores arbitrários (Redis).



Fonte: Elaborado pelo autor (2026).

2.2.2 Colunar

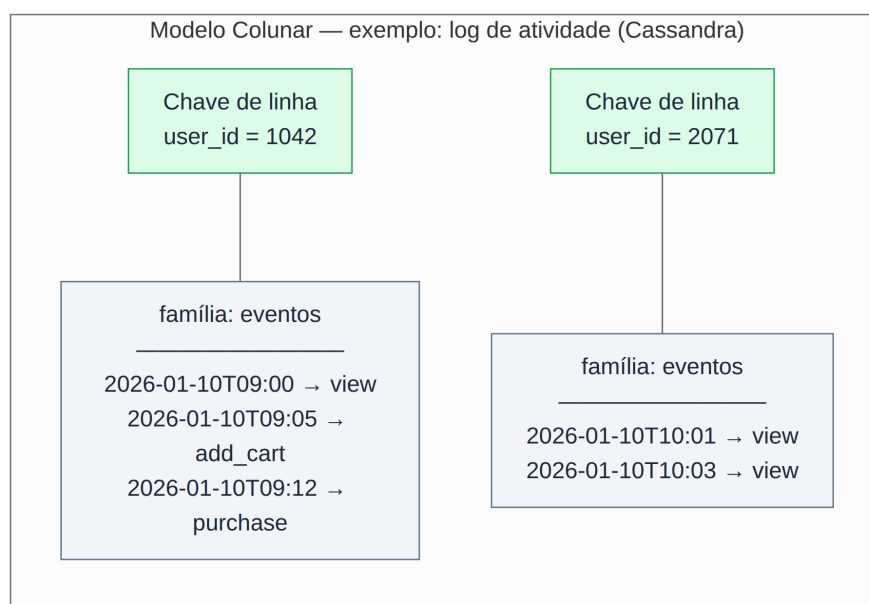
Em bancos de dados colunares, o conjunto de dados consiste em várias linhas, cada uma identificada por uma chave de linha única, semelhante a uma chave primária. Cada linha é composta por uma família (ou conjunto) de colunas, e diferentes linhas podem ter diferentes famílias de colunas. Semelhante aos bancos de dados de chave-valor, a chave de linha funciona como a chave, e o conjunto de famílias de colunas atua como o valor representado pela chave de linha. No entanto, colunas em uma família de colunas consistem em pares atributo-valor que podem ser indexadas. O SGBD Cassandra oferece a funcionalidade adicional de supercolunas, que são criadas agrupando várias colunas.

Normalmente, os dados pertencentes a uma linha são armazenados juntos

no mesmo nó do servidor. No entanto, o Cassandra pode distribuir uma única linha por vários nós de servidor usando chaves de partição compostas. Nos bancos de dados colunares, a configuração das famílias de colunas é tipicamente feita durante a inicialização. No entanto, a definição prévia de colunas não é necessária, oferecendo grande flexibilidade no armazenamento de qualquer tipo de dado.

De forma semelhante aos bancos de dados de chave-valor, qualquer lógica que exija relações deve ser implementada na aplicação cliente, ou seja, essa categoria de banco de dados também não têm ciência de relacionamentos entre dados e controle de integridade referencial. A Figura 4 apresenta um exemplo de *log* de atividade, em que cada chave de linha (*user_id*) agrupa uma família de colunas com eventos indexados por *timestamp*.

Figura 4 – Modelo colunar: chaves de linha e famílias de colunas (Cassandra).

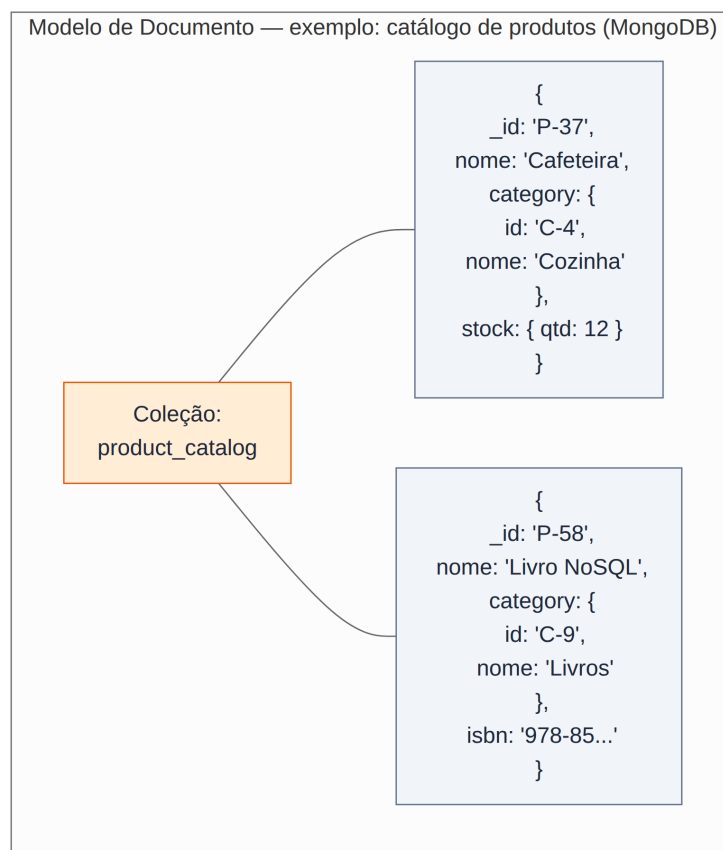


Fonte: Elaborado pelo autor (2026).

2.2.3 Documento

Bancos de dados orientados a documento armazenam dados como documentos autocontidos, em geral em formato JSON ou BSON. Cada documento possui um identificador e um conjunto de campos que podem variar entre documentos de uma mesma coleção, favorecendo esquemas flexíveis e agregados de leitura (*read-optimized*). O SGBD MongoDB é um exemplo popular nesta categoria. Ele define coleções de documentos, índices secundários e *pipelines* de agregação, permitindo consultas ricas sem exigir junções (*joins*) no servidor. A Figura 5 ilustra uma coleção de catálogo de produtos, em que cada documento é autocontido e pode conter subdocumentos aninhados (como *category* e *stock*) e campos distintos entre si.

Figura 5 – Modelo de documento: documentos autocontidos com campos variáveis (MongoDB).



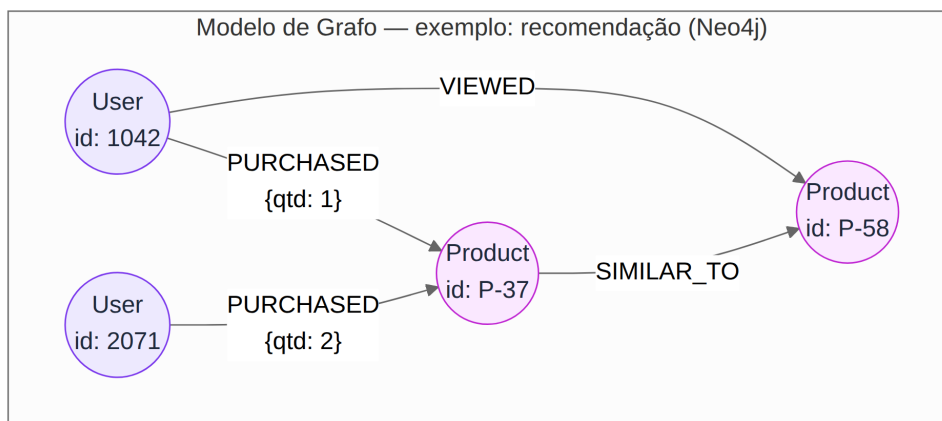
Fonte: Elaborado pelo autor (2026).

2.2.4 Grafo

Bancos de dados em grafo representam dados como **nós** (entidades) e **arestas** (relacionamentos tipados), com propriedades em ambos. Consultas exploram caminhos e vizinhanças de forma natural, o que é útil para recomendação, detecção de fraude e redes sociais. O Neo4j utiliza a linguagem Cypher para MERGE/CREATE de nós e relacionamentos.

Na ferramenta proposta, o destino grafo exige que o usuário declare explicitamente quais colunas do CSV identificam cada rótulo de nó e quais colunas alimentam propriedades do relacionamento (por exemplo, comprador → vendedor com atributos do pedido). A Figura 6 exemplifica um cenário de recomendação, em que nós User e Product são conectados por arestas tipadas (PURCHASED, VIEWED, SIMILAR_TO), algumas portando propriedades próprias.

Figura 6 – Modelo de grafo: nós e arestas tipadas com propriedades (Neo4j).



Fonte: Elaborado pelo autor (2026).

2.3 BANCOS DE DADOS MULTIMODELO E ARQUITETURAS DE PERSISTÊNCIA POLIGLOTA

SGBDs multimodelo integram mais de um paradigma de dados no mesmo motor de gerência de dados (por exemplo, documento + chave-valor + grafo), oferecendo em geral uma interface de acesso unificada — como a linguagem AQL do SGBD ArangoDB, que combina filtros sobre estruturas de documento similares ao JSON e travessias em grafos em uma única abstração de modelo de dados que combina ambos os modelos NoSQL.

A persistência poliglota, por outro lado, combina vários SGBDs heterogêneos, cada um escolhido pela adequação a um caso de uso da aplicação (SADALAGE; FOWLER, 2013). Na literatura recente, essa ideia materializa-se como um SGPD (Sistema de Gerenciamento Poliglota de Dados): coordenação transparente entre tecnologias distintas, com requisitos de modelagem e expressividade de acesso (diferentes padrões de consulta) e capacidade de adaptação quando o *workload* evolui (KIEHN et al., 2022; GLAKE et al., 2022).

Tan et al. (2017) apresentam uma taxonomia útil para distinguir arquiteturas relacionadas: BD federado (fontes homogêneas, interface única), BD multistore (fontes heterogêneas, modelo canônico de consulta), BD *polystore* (fontes heterogêneas, múltiplas interfaces de acesso — exemplificadas por sistemas como o BigDAWG) e abordagens multimodelo (vários modelos no mesmo motor).

A escolha entre essas arquiteturas não é trivial. Roy-Hubara, Shoval e Sturm (2022) propõem critérios para selecionar quais modelos de dados usar em aplicações de persistência poliglota, destacando o *trade-off* entre especialização por SGBD e simplicidade operacional de um motor multimodelo.

O *Polyglot Import CSV* adota explicitamente a linha da persistência poliglota ao invés de multimodelo, conforme detalhado no capítulo 4. O usuário direciona entidades e filtros para PostgreSQL, Redis, MongoDB, Cassandra e Neo4j conforme o padrão de representação e acesso desejados.

3 TRABALHOS RELACIONADOS

Este capítulo apresenta e discute ferramentas oferecidas na indústria por SGBDs e também propostas na literatura acadêmica relacionadas com a importação de dados por SGBDs heterogêneos. A preferência está em dados no formato CSV que devem ser importados por alguma tecnologia de banco de dados, que é o foco deste trabalho.

3.1 IMPORTAÇÃO DE ARQUIVO CSV PARA UM ÚNICO SGBD

Esta seção investiga mecanismos de importação de dados presentes em SGBDs populares na indústria que são considerados neste trabalho.

3.1.1 PostgreSQL

No SGBD PostgreSQL você utiliza o comando `COPY` para realizar importações de dados CSV ([The PostgreSQL Global Development Group, 2026](#)). É necessário especificar a tabela com os nomes das colunas após a cláusula `COPY`. A ordem das colunas deve ser a mesma que aquelas no arquivo CSV, conforme ilustra o exemplo a seguir:

```
COPY nome_da_tabela(col1, col2, col3)
FROM 'C:/caminho_do_arquivo/dados_de_amostra.csv'
DELIMITER ','
CSV HEADER;
```

Caso o arquivo CSV contenha todas as colunas da tabela, você não precisa especificá-las explicitamente:

```
COPY nome_da_tabela
FROM 'C:/caminho_do_arquivo/dados_de_amostra.csv'
DELIMITER ','
CSV HEADER;
```

Você também deve inserir o caminho do arquivo CSV após a palavra-chave `FROM`. Como o formato do arquivo é CSV, você precisa especificar `DELIMITER`, bem como a cláusula `CSV`. A cláusula `HEADER` deve ser especificada para indicar que o arquivo CSV contém um cabeçalho. Quando o comando `COPY` importa dados, ele ignora o cabeçalho do arquivo.

O arquivo CSV deve ser lido diretamente pelo servidor PostgreSQL, não pela aplicação cliente. Portanto, ele deve ser acessível pela máquina do servidor PostgreSQL. Além disso, é necessário ter permissão de superusuário para executar com sucesso o comando `COPY`.

3.1.2 Redis

A maneira preferida de importar uma massa de dados para o Redis é gerar um arquivo de texto contendo o protocolo Redis, em formato bruto, para chamar os comandos necessários para inserir os dados requeridos ([Redis Ltd., 2026](#)).

Por exemplo, se é necessário gerar um grande conjunto de dados com bilhões de chaves no formato “chaveN → ValorN”, deve ser criado um arquivo contendo os seguintes comandos:

```
SET Chave0 Valor0
SET Chave1 Valor1
SET ChaveN ValorN
```

Uma vez criado esse arquivo, a ação restante é alimentá-lo no Redis. No passado, a maneira de fazer isso era usar o netcat com o seguinte comando:

```
(cat dados.txt; sleep 10) | nc localhost 6379
```

Nas versões 2.6 ou posteriores do Redis, o utilitário `redis-cli` suporta um novo modo chamado modo *pipe* que foi projetado para realizar o carregamento em massa. O comando a ser executado neste caso é o seguinte:

```
cat dados.txt | redis-cli --pipe
```

O utilitário `redis-cli` imprime um pequeno resumo. A saída típica é semelhante a esta:

```
All data transferred. Waiting for the last reply...
Last reply received from server.
errors: 0, replies: 1000000
```

A linha de contagem ao final é a informação mais importante do resumo: `errors` indica quantos comandos foram rejeitados pelo servidor, e `replies` indica o total de respostas recebidas. Se `errors: 0`, todos os comandos foram processados com sucesso. Um valor maior que zero em `errors` significa que ao menos um comando falhou — seja por sintaxe inválida no arquivo, por tipo de dado incompatível com a chave existente, ou por qualquer outra razão que o Redis tenha rejeitado.

Vale notar que o `--pipe` não exibe cada resposta individualmente (como um `+OK` por `SET`), o que é intencional: imprimir bilhões de respostas destruiria o ganho de *performance* do modo *pipe* e inundaria o terminal. Em vez disso, o `redis-cli` lê e contabiliza todas as respostas silenciosamente conforme chegam, entregando apenas o agregado final.

Por fim, uma verificação útil: o valor de `replies` deve ser igual ao número de comandos enviados. Uma divergência entre os dois pode indicar que o arquivo de entrada estava truncado ou malformado, mesmo que `errors` apareça como zero.

3.1.3 MongoDB

No SGBD MongoDB, o processo de importação de dados em formato CSV é realizado através do utilitário `mongoimport` ([MongoDB, Inc., 2026](#)). Sua sintaxe básica exige que se informe o banco de dados de destino através do parâmetro `--db`, seguido do nome da coleção que receberá os dados, especificado em `--collection`. Caso esse último parâmetro seja omitido, o nome da coleção será automaticamente inferido a partir do nome do arquivo CSV fornecido.

```
mongoimport --db nome_do_db --collection nome_da_colecao \  
--type csv --headerline --ignoreBlanks \  
--file caminho/do/arquivo.csv
```

O tipo do arquivo de entrada deve ser declarado explicitamente através do parâmetro `--type`, que neste caso recebe o valor `csv`. Para que o `mongoimport` reconheça os nomes dos campos a partir do próprio arquivo, utiliza-se o parâmetro `--headerline`, que instrui o utilitário a tratar o conteúdo da primeira linha do CSV como o cabeçalho dos campos. Opcionalmente, o parâmetro `--ignoreBlanks` pode ser utilizado para que valores em branco presentes no arquivo sejam ignorados durante a importação, evitando que campos vazios sejam persistidos na coleção. Por fim, o caminho do arquivo CSV a ser importado é fornecido através do parâmetro `--file`.

3.1.4 Cassandra

A importação de dados CSV para o Cassandra utiliza o comando `sstableloader` e envolve vários passos ([The Apache Software Foundation, 2026](#)). Inicialmente, é necessário se certificar que o arquivo CSV está no formato adequado para o Cassandra. Isso geralmente envolve garantir que os tipos de dados e as colunas correspondem ao esquema da tabela no Cassandra.

O `sstableloader` requer que os dados estejam em um formato específico chamado `SSTable`. Para converter o CSV em `SSTables`, você pode usar o comando `cqlsh` fornecido pelo Cassandra. Um exemplo de uso é o seguinte:

```
cqlsh -e "COPY keyspace_name.table_name TO 'output_directory';"
```

Na sequência, é possível usar o `sstableloader` para carregar os dados. A sintaxe do comando é a seguinte:

```
sstableloader -d <hostname> -u <username> -pw <password> \  
output_directory/keyspace_name/table_name
```

- `<hostname>`: O endereço do nó Cassandra.
- `<username>`: O nome de usuário, se a autenticação estiver habilitada.
- `<password>`: A senha correspondente.

Este comando move os dados do diretório de saída para o Cassandra.

3.1.5 Neo4j

Usar `neo4j-admin` para importar grandes conjuntos de dados no Neo4j envolve alguns passos (Neo4j, Inc., 2026). Esta ferramenta de linha de comando é projetada para importações em massa eficientes.

Certifique-se de que seus arquivos CSV estejam estruturados corretamente e sigam o formato necessário para nós e relacionamentos. Crie um banco de dados não povoado:

```
neo4j-admin create-db --database=nome_do_seu_banco_de_dados \
--from=diretorio_com_arquivos_csv
```

Substitua `nome_do_seu_banco_de_dados` pelo nome desejado para o novo banco de dados. A opção `--from` especifica o diretório onde estão localizados seus arquivos CSV. Execute o comando de importação com `neo4j-admin`. O comando exato depende do seu modelo de dados, mas pode se parecer com isto:

```
neo4j-admin import --database=nome_do_seu_banco_de_dados \
--mode=csv \
--nodes:Rotulo1 nos.csv \
--nodes:Rotulo2 nos2.csv \
--relationships:TIPO_RELACIONAMENTO relacionamentos.csv
```

Substitua *Rotulo1*, *Rotulo2*, *nos.csv*, *nos2.csv*, *TIPO_RELACIONAMENTO* e *relacionamentos.csv* pelos seus rótulos específicos e nomes de arquivo.

3.2 IMPORTAÇÃO DE ARQUIVO CSV PARA SGBD MULTIMODELO

Ferramentas comerciais e de código aberto tratam de **migração** ou **modelagem** entre tecnologias heterogêneas, embora raramente com o foco deste TCC (um único CSV largo + regras declarativas + cinco paradigmas).

- **ArangoDB** é um SGBD multimodelo nativo (documento/chave-valor/grafos) com `arangoimport` para CSV; a importação é para **um** motor multimodelo, não para cinco *backends* independentes.
- `mongoimport` / `COPY` / `redis-cli --pipe` / `cqlsh` / `neo4j-admin` (seção 3.1) cobrem importação **para um único** produto por vez.
- **dbcrossbar** (DBCROSSBAR..., 2024) copia dados tabulares entre PostgreSQL, BigQuery, armazenamento em nuvem e CSV, incluindo conversão de esquemas; a ênfase é *pipeline* de migração, não mapeamento semântico de entidades aninhadas com filtros por valor.

- **Hackolade** ([Hackolade, 2024](#)) oferece *polyglot data modeling* com engenharia de esquema para dezenas de alvos; trata-se de **modelagem visual** e geração de artefatos, e não de um utilitário CLI único para importar um CSV operacional com validação prévia de filtros.

3.3 CONSIDERAÇÕES SOBRE OS TRABALHOS RELACIONADOS

A literatura sobre **metamodelos unificados** para SQL e NoSQL (por exemplo, U-Schema ([BERLANGA et al., 2021](#))) e sobre **evolução de esquema** em ambientes heterogêneos ([CHILLÓN et al., 2023](#); [CHILLÓN et al., 2024](#)) informa decisões de validação e de representação de entidades no JSON de configuração, mas não substitui uma ferramenta de *bulk import* orientada a CSV.

Surveys recentes convergem em desafios abertos da gestão poliglota de dados: migração automatizada entre *stores* quando o *workload* muda, planejamento de consultas entre sistemas, gestão de esquemas multi-modelo e preservação das propriedades não funcionais de cada SGBD ([KIEHN et al., 2022](#); [GLAKE et al., 2022](#)). [Roy-Hubara, Shoval e Sturm \(2022\)](#) tratam da **seleção de SGBDs** por fragmento de aplicação; [Silva, Bornia e Mello \(2024\)](#) mapeiam o estado da arte em **modelagem poliglota**. Nenhum desses trabalhos implementa, contudo, um utilitário declarativo que receba um CSV operacional único e o materialize simultaneamente em cinco paradigmas distintos com validação estática prévia.

Abordagens recentes reforçam essa tendência. [Pereira, Guerra e Rosa \(2025\)](#) propõem um *framework* de persistência poliglota **transparente**, com uma API unificada e extensível a diferentes tipos de banco, enquanto [Sachdeva e Vasava \(2024\)](#) exploram a combinação de bancos **multimodelo** e persistência poliglota para acomodar a variedade dos dados. Ambos os trabalhos atuam, porém, na **camada de acesso da aplicação** — oferecendo uma interface única para ler e gravar em múltiplos SGBDs em tempo de execução —, e não na **carga inicial declarativa** a partir de um CSV operacional, que é o foco da *Polyglot Import CSV*.

Sistemas *polystore* como o BigDAWG ([DUGGAN et al., 2017](#)) e *middlewares* de portabilidade em nuvem ([WANG et al., 2020](#)) tratam de **consultas federadas** e movimentação de dados em ecossistemas complexos. [Tan et al. \(2017\)](#) classificam essas soluções (federado, multistore, polystore, poliglota) e avaliam transparência, autonomia e heterogeneidade — eixos nos quais importadores nativos (seção 3.1) e ferramentas de migração (seção 3.2) também divergem, mas sem convergir para um *pipeline* único de ingestão.

Ferramentas como **dbcrossbar** ([DBCROSSBAR. . . , 2024](#)) e **Hackolade** ([Hackolade, 2024](#)) aproximam-se do domínio de integração de dados, porém com foco

em *pipelines* de migração ou modelagem visual, respectivamente — não em regras declarativas sobre entidades aninhadas, filtros por valor (*each*) e validação por JSON Schema antes de qualquer conexão com SGBDs.

Lacuna identificada: a combinação de (i) um CSV largo como fonte operacional, (ii) configuração JSON validada estaticamente, (iii) materialização distinta por paradigma (relacional, chave-valor, documento, colunar, grafo) e (iv) modo *dry-run* para revisão prévia. O *Polyglot Import CSV* ocupa essa lacuna como ferramenta de **bootstrap de persistência poliglota** — carga inicial e reproduzível de dados — enquanto *middlewares* e *polystores* assumem dados já residentes ou consultas em tempo de execução. Trabalhos futuros podem integrar a ferramenta a mecanismos de migração adaptativa descritos por [Kiehn et al. \(2022\)](#) e [Glake et al. \(2022\)](#).

4 A PROPOSTA DA FERRAMENTA POLYGLOTIMPORTCSV

4.1 VISÃO GERAL

A ferramenta **PolyglotImportCSV** (pacote Python `polyglot-import-csv`) lê um arquivo CSV e **dois arquivos de configuração JSON** — um de importação (mapeamento) e um de conexão dos SGBDs — cada um validado por um **JSON Schema** embutido. A CLI executa, em sequência:

1. Carregamento do CSV (como texto, para evitar erros de *parse* em campos com + em *timestamps*).
2. Inferência de *kinds* de coluna (inteiro, *float*, data/hora, texto) para validar operadores de filtro.
3. Validação cruzada: colunas referenciadas existem no CSV; relacionamentos PostgreSQL e Neo4j referenciam entidades conhecidas; chaves de partição Cassandra estão declaradas quando necessário.
4. Para cada *backend* configurado: aplicação de filtros (incluindo o operador *each* para particionar por valor distinto), materialização de entidades e importação (ou apenas resumo em `--dry-run`).

Comando principal:

```
python -m polyglotimportcsv caminho/dados.csv \
  --config caminho/import_config.json \
  --sgbd-config caminho/sgbd_config.json \
  [--dry-run] \
  [--create-schema / --no-create-schema] \
  [--only postgres,redis]
```

4.2 FORMATO DE CONFIGURAÇÃO (JSON SCHEMA)

A configuração é dividida em **dois arquivos JSON**, cada um validado estaticamente por um **JSON Schema** próprio embutido em `src/polyglotimportcsv/schemas/` (rascunho 2020-12):

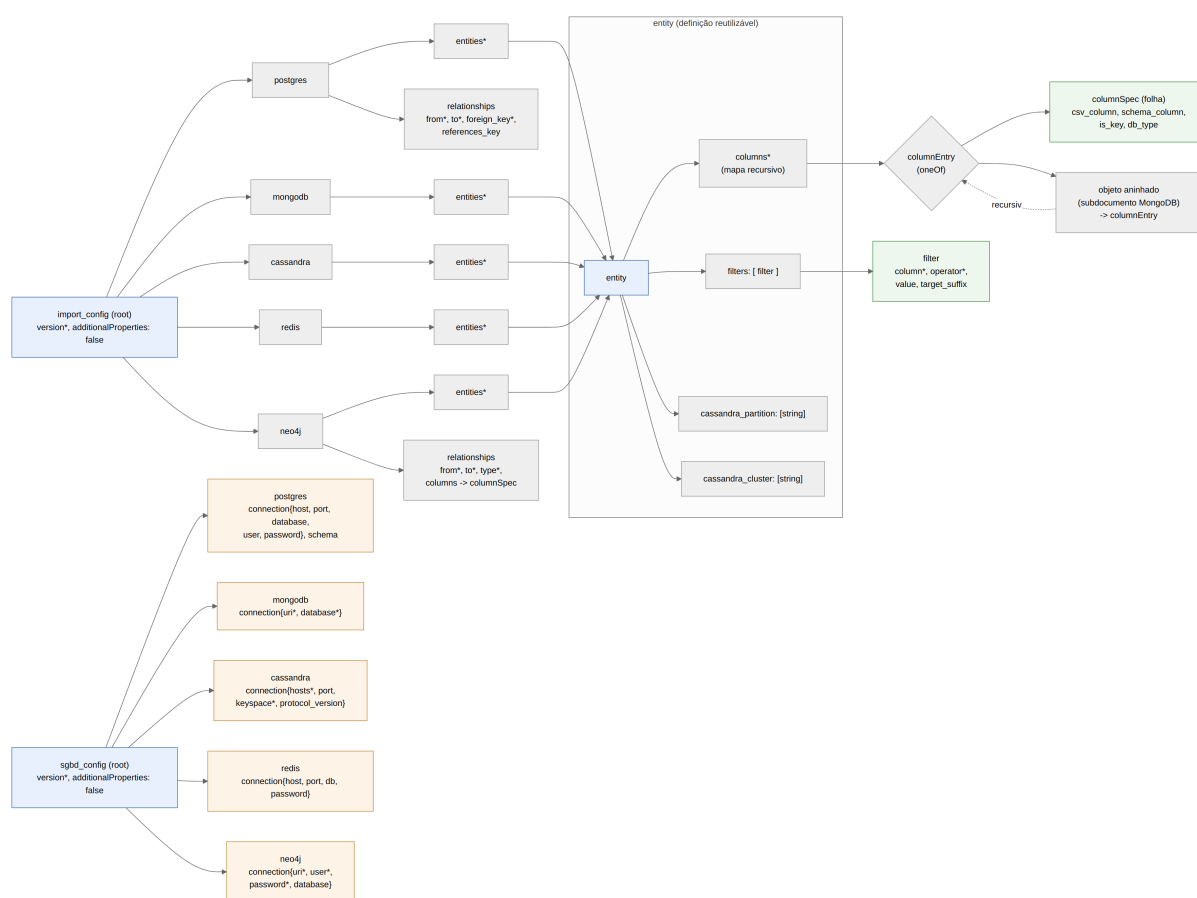
- `import_config.json` — o **mapeamento** de entidades, relacionamentos e colunas do CSV para cada SGBD;
- `sgbd_config.json` — a **configuração de conexão** de cada SGBD, isto é, quais bancos estão disponíveis e como alcançá-los.

Essa separação atende a duas preocupações distintas: o *mapeamento* muda conforme o domínio dos dados; a *conexão* muda conforme o ambiente (desenvolvimento, testes, produção). Um SGBD só pode ser referenciado no arquivo de importação se estiver **declarado** no arquivo de conexão — esta e as demais regras de validação são detalhadas na seção 4.2.8. A representação completa de ambos os schemas na forma serializada JSON Schema 2020-12 encontra-se no Apêndice A.

4.2.1 Visão Geral da Estrutura

Antes de detalhar cada *backend* individualmente, esta subseção apresenta a estrutura dos dois schemas como um todo, fornecendo um mapa mental que orienta a leitura das subseções seguintes. A Figura 7 sintetiza graficamente os dois JSON Schemas: o de importação (*import_config*) no topo e o de conexão (*sgbd_config*) na base, ambos à esquerda. O diagrama torna desnecessária a leitura do schema serializado integral — mantido no Apêndice A — para a compreensão geral da configuração.

Figura 7 – Estrutura dos dois JSON Schemas de configuração (importação e conexão).



Fonte: Elaborado pelo autor (2026).

Raízes simétricas. Ambos os schemas partilham a mesma raiz: um campo `version` obrigatório e um bloco opcional por *backend* (`postgres`, `mongodb`, `cassandra`,

redis, neo4j), com `additionalProperties: false` rejeitando chaves desconhecidas. A diferença está no conteúdo de cada bloco: no arquivo de importação ele descreve o *que* mapear; no de conexão, *onde* gravar.

O arquivo de importação e a definição reutilizável `entity`. Seguindo as setas a partir de `import_config`, cada bloco de *backend* expande-se em um mapa `entities` (e, para PostgreSQL e Neo4j, também em `relationships`). O ponto central do diagrama — e da economia de vocabulário do schema — é que **todas as entidades, de todos os *backends*, convergem para uma única definição reutilizável**, rotulada `entity`. Em vez de cinco formatos distintos, há apenas um: toda entidade possui um mapa `columns` (o mapeamento de colunas) e, opcionalmente, `filters` (predicados sobre as linhas do CSV) e os campos específicos do Cassandra (`cassandra_partition` e `cassandra_cluster`).

O mapeamento recursivo de colunas. À direita do diagrama, `columns` remete a `columnEntry`, definido como uma escolha (`oneOf`) entre duas formas: (i) uma **folha** — o `columnSpec`, que liga uma coluna do CSV a um atributo do destino por meio dos campos `csv_column`, `schema_column`, `is_key` e `db_type`; ou (ii) um **objeto aninhado**, que contém novos `columnEntry`. Essa recursão é o que permite, com um único mecanismo, tanto o mapeamento plano (PostgreSQL, Cassandra, Redis) quanto os subdocumentos do MongoDB. As arestas tipadas do Neo4j e as chaves estrangeiras do PostgreSQL aparecem em `relationships`, reutilizando o mesmo `columnSpec` para eventuais propriedades.

O arquivo de conexão. Na base do diagrama, cada bloco de *backend* de `sgbd_config` carrega apenas um descritor `connection` com os parâmetros de acesso pertinentes àquela tecnologia (por exemplo, `uri` para MongoDB e Neo4j; `hosts` e `keyspace` para Cassandra). É uma estrutura deliberadamente rasa: indica *onde* alcançar cada SGBD, sem nenhuma informação de mapeamento.

As subseções seguintes percorrem essa estrutura na ordem em que o diagrama é lido: primeiro a raiz e os elementos comuns a todos os *backends* (`columnSpec`, `entity` e filtros) e, em seguida, as particularidades de cada SGBD, ilustradas com fragmentos concretos de configuração.

4.2.2 Raiz, Elementos Comuns e Reforma de Simplificação

Como antecipado na visão geral, ambos os arquivos compartilham a mesma raiz: um campo `version` obrigatório (inteiro ≥ 1) e um bloco opcional por *backend*. A cláusula `additionalProperties: false` na raiz **rejeita** qualquer propriedade não prevista, impedindo que erros de digitação como `postgress` passem despercebidos.

`columnSpec`. Cada **folha** do mapa `columns` descreve como um valor do CSV alimenta um atributo no destino. Quatro campos opcionais compõem esse mapeamento,

conforme a Tabela 1.

Tabela 1 – Campos de mapeamento de coluna (columnSpec).

Campo	Tipo	Obrigatório	Descrição
csv_column	<i>string</i> ou inteiro (≥ 0)	não	Cabeçalho CSV ou índice da coluna (base 0). Padrão: chave JSON.
schema_column	<i>string</i>	não	Nome do atributo no SGBD destino. Padrão: chave JSON.
is_key	booleano	não	Chave primária, chave de MERGE (Neo4j) ou chave Redis.
db_type	<i>string</i>	não	Tipo SQL/CQL para geração de DDL.

Fonte: Elaborado pelo autor (2026).

A forma mínima ("campo": {}) é usada quando o nome no CSV, no schema e na chave JSON coincidem. csv_column e schema_column existem porque **nem sempre esses três nomes coincidem**: o campo csv_column indica a origem no CSV; schema_column, o destino no SGBD.

entity. Uma entidade representa um alvo lógico: tabela (PostgreSQL/Cassandra), coleção (MongoDB), rótulo de nó (Neo4j) ou entidade Redis. Campos obrigatórios: columns (mapa recursivo de columnSpec). Opcionais: filters (predicados sobre o CSV), cassandra_partition e cassandra_cluster (apenas Cassandra).

Filtros. Cada item exige column e operator (==, !=, >, <, >=, <=, in, not_in, each). O operador each particiona a entidade por valores distintos da coluna, gerando múltiplos alvos com sufixo opcional.

Reforma de simplificação. Versões anteriores acumulavam campos redundantes, mantidos por retrocompatibilidade. Esses campos foram removidos conforme a Tabela 2, reduzindo o vocabulário ao mínimo necessário.

Tabela 2 – Campos removidos na reforma de simplificação do schema.

Campo removido	Substituto	Justificativa
db_column	schema_column	Sinônimo de renomeação no destino; redundante.
alias_db	schema_column	Segundo sinônimo da mesma operação de renomeação.
nested	chaves aninhadas em columns	O aninhamento de subdocumentos MongoDB já é expresso recursivamente pelo mapa columns.

Fonte: Elaborado pelo autor (2026).

4.2.3 PostgreSQL

Arquivo de importação. O bloco postgres contém entities (entidades planas — aninhamento de columns é rejeitado) e, opcionalmente, relationships para declarar chaves estrangeiras. O campo db_type em cada columnSpec alimenta a geração de DDL (CREATE TABLE) quando --create-schema está ativo.

Listing 4.1 – Fragmento de importação PostgreSQL (e-commerce).

```
"postgres": {
  "entities": {
    "products": {
      "columns": {
        "product_id": { "is_key": true, "db_type": "INTEGER" },
        "name": { "schema_column": "product_name", "db_type": "VARCHAR(200)" },
        "price": { "db_type": "NUMERIC(10,2)" }
      },
      "filters": [{ "column": "action", "operator": "=", "value": "stock" }]
    }
  },
  "relationships": {
    "product_category": {
      "from": "products",
      "to": "categories",
      "foreign_key": "category_id",
      "references_key": "category_id"
    }
  }
}
```

Arquivo de conexão. O bloco postgres contém connection (host, port, database, user, password) e o campo schema (padrão public), que define o schema

PostgreSQL de destino das tabelas.

Listing 4.2 – Fragmento de conexão PostgreSQL.

```
"postgres": {
  "connection": {
    "host": "localhost", "port": 5432,
    "database": "ecommerce_db",
    "user": "pguser", "password": "secret"
  },
  "schema": "public"
}
```

4.2.4 MongoDB

Arquivo de importação. O bloco `mongodb` contém apenas `entities`. O diferencial do MongoDB é o suporte a **aninhamento de columns**: chaves JSON aninhadas produzem subdocumentos BSON, sem necessidade de nenhum campo adicional — o próprio mapa recursivo expressa a hierarquia.

Listing 4.3 – Fragmento de importação MongoDB com subdocumentos.

```
"mongodb": {
  "entities": {
    "product_catalog": {
      "columns": {
        "product_id": { "is_key": true },
        "category": {
          "category_id": {},
          "category_name": {}
        },
      },
      "stock": {
        "quantity_available": {},
        "last_restock_date": {}
      }
    },
    "filters": [{ "column": "action", "operator": "=", "value": "stock" }]
  }
}
```

Arquivo de conexão. O bloco `mongodb` contém `connection` com `uri` (string de conexão MongoDB, incluindo usuário e senha quando necessário) e `database` (nome do banco de destino).

Listing 4.4 – Fragmento de conexão MongoDB.

```
"mongodb": {
  "connection": {
    "uri": "mongodb://localhost:27017",
    "database": "ecommerce_db"
  }
}
```

4.2.5 Apache Cassandra

Arquivo de importação. O bloco `cassandra` contém `entities` planas. Dois campos específicos controlam a chave primária composta do Cassandra: `cassandra_partition` (lista de colunas CSV que formam a chave de partição) e `cassandra_cluster` (colunas de agrupamento). A ausência de `cassandra_partition` é detectada na validação cruzada quando não há nenhuma coluna marcada com `is_key`.

Listing 4.5 – Fragmento de importação Cassandra com chave composta.

```
"cassandra": {
  "entities": {
    "user_activity_log": {
      "columns": {
        "user_id": {},
        "timestamp": { "schema_column": "event_time" },
        "action": {},
        "product_id": {}
      },
      "cassandra_partition": ["user_id"],
      "cassandra_cluster": ["timestamp"]
    }
  }
}
```

O campo `schema_column` em `timestamp` renomeia a coluna CSV para `event_time` na tabela CQL — exemplo concreto de como `csv_column` e `schema_column` atuam independentemente.

Arquivo de conexão. O bloco `cassandra` contém `connection` com `hosts` (lista de endereços de nós), `port` (padrão 9042), `keyspace` e `protocol_version` opcional.

Listing 4.6 – Fragmento de conexão Cassandra.

```
"cassandra": {
  "connection": {
    "hosts": ["localhost"],
    "port": 9042,
    "keyspace": "ecommerce_ks"
  }
}
```

4.2.6 Redis

Arquivo de importação. O bloco `redis` contém `entities`. A restrição fundamental é que **cada entidade deve ter exatamente uma coluna marcada com `is_key`**: esse valor se torna a chave Redis (prefixada pelo nome da entidade), e todos os demais campos da linha compõem o valor JSON armazenado.

Listing 4.7 – Fragmento de importação Redis.

```

"redis": {
  "entities": {
    "shopping_cart": {
      "columns": {
        "user_id":      { "is_key": true },
        "product_id":   {},
        "quantity":     {},
        "added_at":     {}
      },
      "filters": [{ "column": "action", "operator": "=", "value": "add_to_cart" }]
    }
  }
}

```

Arquivo de conexão. O bloco redis contém connection com host, port (padrão 6379), db (índice do banco, padrão 0) e password opcional.

Listing 4.8 – Fragmento de conexão Redis.

```

"redis": {
  "connection": {
    "host": "localhost",
    "port": 6379,
    "db": 0
  }
}

```

4.2.7 Neo4j

Arquivo de importação. O bloco neo4j contém entities (nós) e relationships (arestas tipadas). Cada entidade declara um rótulo (*label*) de nó e deve ter exatamente uma coluna is_key, usada como predicado de identificação no MERGE Cypher. As arestas em relationships declaram from (rótulo de origem), to (rótulo de destino), type (tipo da aresta em maiúsculas) e um mapa columns opcional com propriedades da aresta.

Listing 4.9 – Fragmento de importação Neo4j (nós e aresta).

```

"neo4j": {
  "entities": {
    "User": { "columns": { "user_id": { "is_key": true } } },
    "Product": { "columns": { "product_id": { "is_key": true } } }
  },
  "relationships": {
    "PURCHASED": {
      "from": "User",
      "to": "Product",
      "type": "PURCHASED",
      "columns": {
        "order_number": { "is_key": true },
        "quantity": {}
      },
      "filters": [{ "column": "action", "operator": "=", "value": "purchase" }]
    }
  }
}

```

```
}  
}
```

Arquivo de conexão. O bloco neo4j contém connection com uri (bolt://...), user, password e database opcional (padrão neo4j).

Listing 4.10 – Fragmento de conexão Neo4j.

```
"neo4j": {  
  "connection": {  
    "uri":      "bolt://localhost:7687",  
    "user":     "neo4j",  
    "password": "secret",  
    "database": "neo4j"  
  }  
}
```

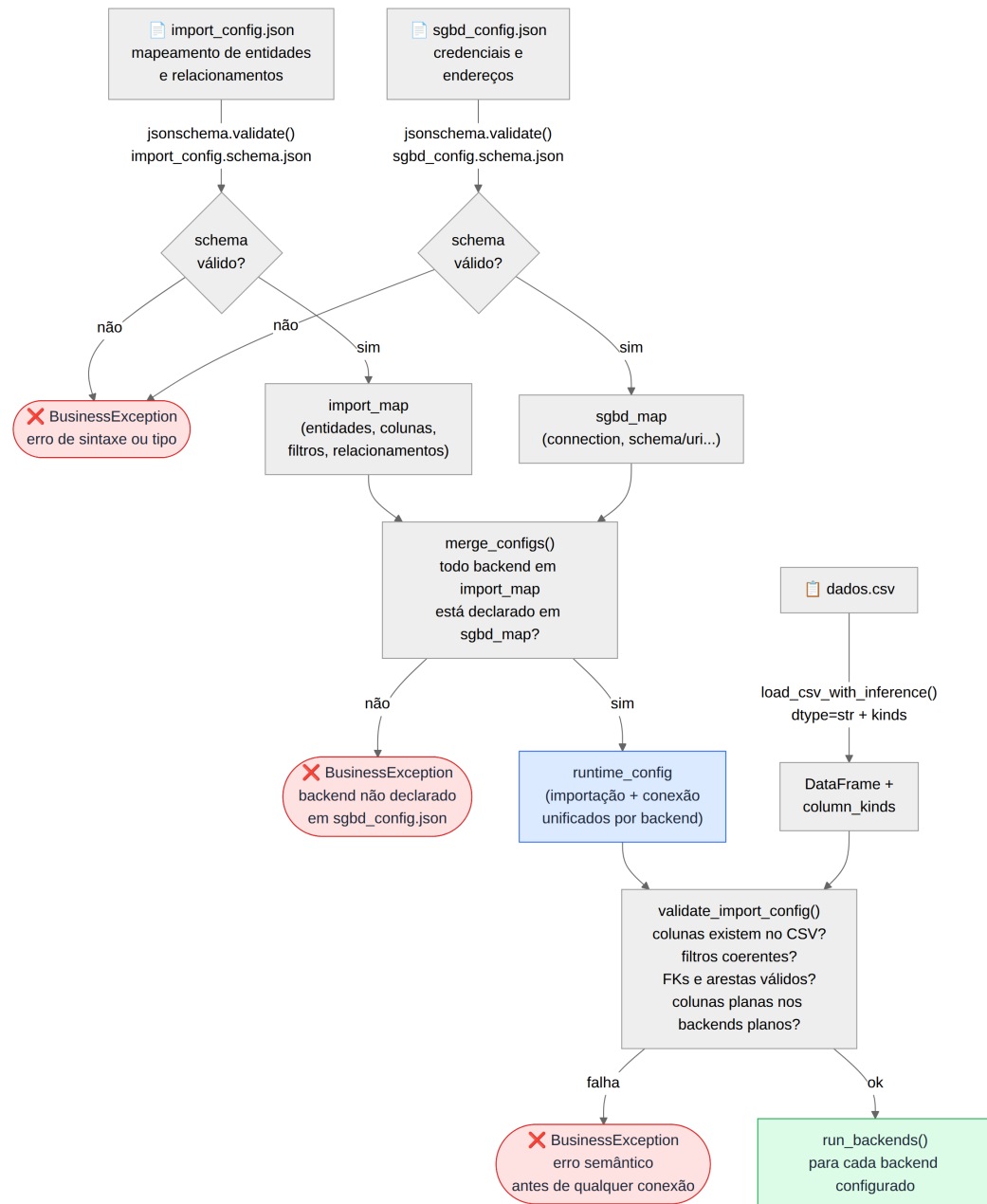
4.2.8 Validação, Consistência entre Arquivos e Fluxo de Carregamento

A validação ocorre em **três camadas** antes de qualquer conexão com os SGBDs:

1. **JSON Schema** — cada arquivo é validado contra o seu schema (sintaxe, tipos e propriedades permitidas):
 - import_config.schema.json;
 - sgbd_config.schema.json.
2. **Consistência entre arquivos** (config_parser.merge_configs) — todo *backend* referenciado no arquivo de importação deve estar declarado no arquivo de conexão.
3. **Validação cruzada CSV** (validation.validate_import_config) — colunas referenciadas existem no CSV; filtros coerentes; FKs PostgreSQL e arestas Neo4j referenciam entidades válidas; *backends* planos não usam columns aninhados; índices csv_column dentro do intervalo.

Qualquer falha levanta BusinessException e **impede** o início da importação. A Figura 8 apresenta o fluxo completo de carregamento, com os pontos de falha de cada camada e a convergência dos dois arquivos em um único runtime_config, ao qual o CSV se junta apenas na validação semântica (camada 3). O diagrama evidencia por que a separação em dois arquivos isola falhas: um sgbd_config.json inválido é detectado independentemente do import_config.json, sem necessidade de ler o CSV.

Figura 8 – Fluxo de carregamento e validação dos dois arquivos de configuração.



Fonte: Elaborado pelo autor (2026).

4.3 ALGORITMO DE EXECUÇÃO DA IMPORTAÇÃO

O núcleo da ferramenta está em `runner.run_import`. A seguir descreve-se o fluxo de alto nível desde a linha de comando até a persistência em cada SGBD.

4.3.1 Fases do Algoritmo

1. **Entrada** — caminho do CSV, caminhos dos dois JSON de configuração (importação e conexão) e *flags*: `--dry-run`, `--create-schema` / `--no-create-schema`,

--only (lista de *backends*).

2. **Carregar configuração** — `load_config`: *parse* dos dois JSON + `jsonschema.validate` de cada um contra o seu schema embutido + verificação de consistência entre arquivos (*backends* declarados) e mesclagem das conexões.
3. **Carregar CSV** — `load_csv_with_inference`: leitura com `dtype=str` (evita erros de *parse* em *timestamps* com +) e inferência de *kind* por coluna (`integer`, `float`, `datetime`, `string`, `empty`).
4. **Validar config × CSV** — `validate_import_config`: checagens semânticas descritas na seção 4.2.5.
5. **Para cada *backend*** presente na configuração e não filtrado por --only:
 - **Para cada entidade** declarada em `entities`:
 - Aplicar filtros (exceto operador `each`) sobre o `DataFrame` completo.
 - Expandir partições `each` (opcional) — uma entidade lógica pode gerar várias tabelas/coleções com sufixo.
 - **Materializar** registros conforme o paradigma do *backend*:
 - * PostgreSQL / Cassandra: `flatten_entity_dataframe` — seleção, renomeação, deduplicação por `is_key`.
 - * MongoDB: `mongo_document_from_row` — documento recursivo a partir de `columns` aninhados.
 - * Redis: `redis_payload_from_row` — par (chave, valor JSON).
 - * Neo4j: propriedades de nós; depois `MERGE` de arestas a partir de `relationships`.
 - Conectar ao SGBD (omitido em --dry-run).
 - Criar schema se --create-schema (DDL PostgreSQL/Cassandra).
 - Persistir (`INSERT`, `insert_many`, `SET`, `MERGE Cypher`, etc.).
6. **Saída** — linhas de *log* com contagens por entidade/*backend*.

4.3.2 Pseudocódigo

```
funcao run_import(csv_path, config_path, sgbd_config_path, dry_run,
  create_schema, only):
  config <- load_config(config_path, sgbd_config_path)
  df, kinds <- load_csv_with_inference(csv_path)
  validate_import_config(config, df, kinds)
  para cada backend em (postgres, mongodb, cassandra, redis, neo4j
  ):
    se backend nao esta em config ou nao esta em only: continuar
    linhas <- run_{backend}_import(config[backend], df, kinds,
                                  dry_run, create_schema)
  registrar linhas no log
```

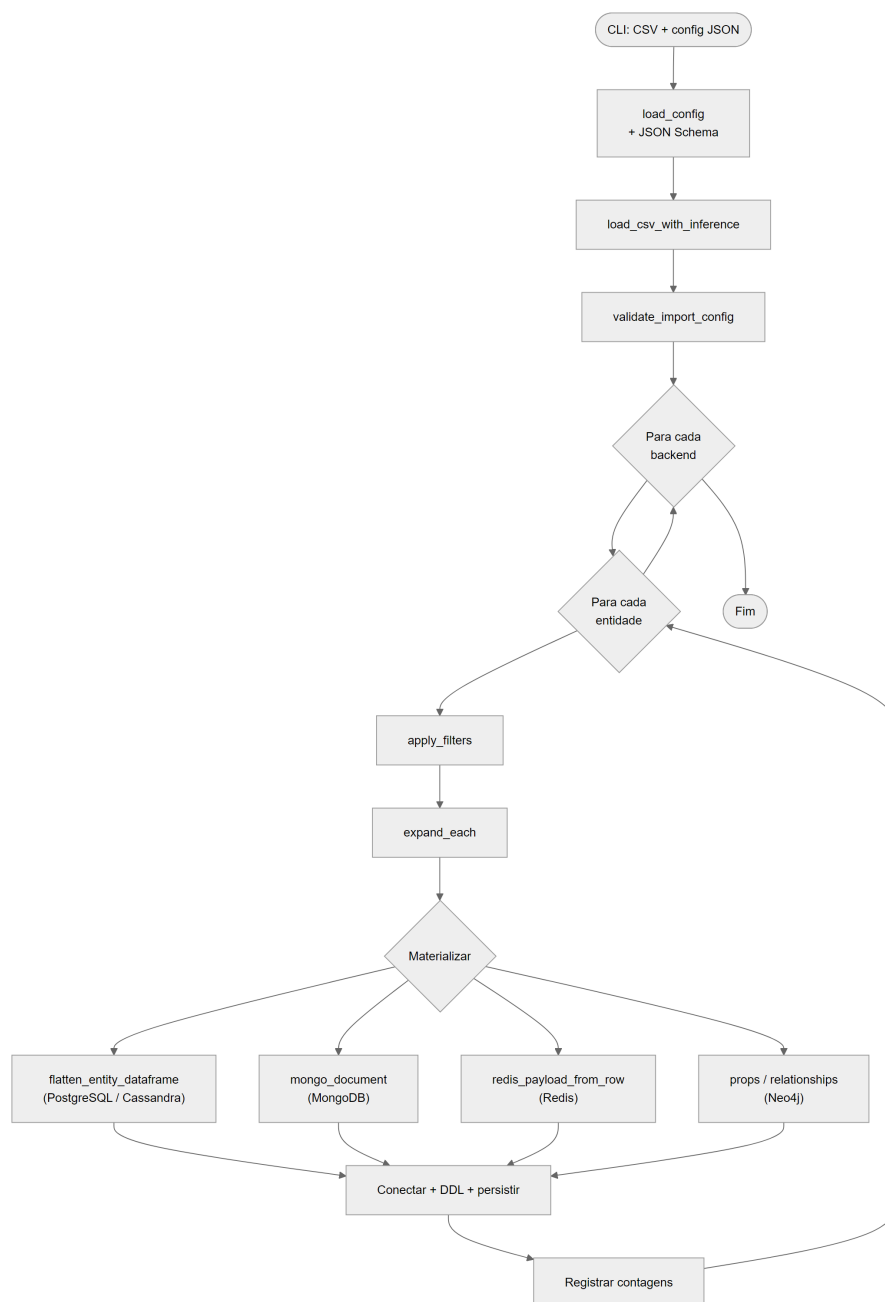
Dentro de cada `run*_import`, para cada entidade:

```
df_filtrado <- apply_filters(df, filtros_sem_each)
para cada (nome_particao, df_parte) em expand_each(...):
  registros <- materializar(df_parte, entidade, backend)
  se nao dry_run: conectar, DDL se necessario, gravar
  registros
```

4.3.3 Diagrama do Fluxo

A Figura 9 resume o *pipeline* de execução descrito acima. O diagrama-fonte Mermaid encontra-se em `docs-tcc/images/figure9-import-algorithm.mmd` para futuras revisões.

Figura 9 – Algoritmo de execução da importação Polyglot Import CSV.



Fonte: Elaborado pelo autor (2026).

4.3.4 Exemplo: Uma Linha action=stock no E-commerce

Considere uma linha do `ecommerce_join.csv` com `action = stock`, contendo `product_id`, `category_id`, `quantity_available`, `timestamp`, etc.

Tabela 3 – Materialização de uma linha `action=stock` nos diferentes backends.

Backend	Entidade	Filtro	Saída materializada
PostgreSQL	inventory	<code>action == stock</code>	Linha plana: <code>product_id</code> , <code>quantity_available</code> , <code>last_restock_date</code> , <code>price</code> .
MongoDB	product_catalog	<code>action == stock</code>	Documento com campos de produto e subobjetos <code>category</code> e <code>stock</code> aninhados.
Cassandra	user_activity_log	nenhum (todas as linhas)	Registro com timestamp renomeado para <code>event_time</code> ; PK composta (<code>user_id</code> , <code>timestamp</code>).
Redis	—	—	Entidades Redis usam outros filtros; esta linha não alimenta Redis.
Neo4j	nó Product	<code>action == stock</code>	Nó :Product com <code>product_id</code> como chave de MERGE.

Fonte: Elaborado pelo autor (2026).

Assim, um **único CSV largo** alimenta destinos heterogêneos: tabelas relacionais filtradas por tipo de evento, documentos aninhados, *log* colunar com renomeação de colunas e nós de grafo — materializando a persistência poliglota descrita no capítulo 2.

4.4 VALIDAÇÃO E EXECUÇÃO

Erros de configuração levantam `BusinessException` antes de qualquer conexão. O modo `--dry-run` lista contagens por entidade sem contatar os SGBDs. O *driver* Apache Cassandra é importado de forma tardia para permitir `--dry-run` em ambientes onde a extensão C do *driver* não está disponível (por exemplo, versões recentes do Python).

Testes automatizados (pytest) cobrem validação de schema, separação e mesclagem dos dois arquivos de configuração, mapeamento de colunas (`csv_column`, `schema_column`, aninhamento), filtros e *smoke tests* de validação + *dry-run*. O arquivo `docker-compose.yml` na raiz define os contêineres de PostgreSQL, Redis, MongoDB, Cassandra e Neo4j para testes de integração manuais; o script `run_example.sh` lê o `srgb_config.json` e **sobe apenas os SGBDs ali declarados**, em seguida orquestra um *dry-run* e a importação real com `--create-schema`.

4.4.1 Evidência de Execução (Cenário E-commerce)

O arquivo `data/ecommerce/ecommerce_join.csv` contém 32 linhas de eventos (`stock`, `purchase`, `add_to_cart`, `select_product`). Usando `import_config.json` e `srgb_config.json` com a *flag* `--dry-run`, a ferramenta reporta as contagens da Tabela 4 sem abrir conexões com os SGBDs — útil para revisar o mapeamento antes da carga.

Tabela 4 – Contagens reportadas pelo modo `--dry-run` no cenário e-commerce.

Backend	Destino	Registros (após filtros/deduplicação)
PostgreSQL	categories	8
PostgreSQL	products	8
PostgreSQL	inventory	8
PostgreSQL	orders	8
MongoDB	product_catalog	8 documentos
Cassandra	user_activity_log	32 linhas (todas as ações)
Redis	shopping_cart	8
Redis	user_session	8
Neo4j	nó User	8
Neo4j	nó Product	8
Neo4j	aresta PURCHASED	conforme linhas purchase

Fonte: Elaborado pelo autor (2026).

Com `docker compose up -d` e `run_example.sh` (ou importação sem `--dry-run`), os mesmos volumes são persistidos nos cinco *backends* configurados, materializando o cenário poliglota da seção 2.1.

5 ATIVIDADES FUTURAS

- **Avaliação de desempenho** das importações com diferentes volumes de dados CSV e configurações, comparando importações simples e complexas para os diferentes modelos de dados destino.
- **Interface gráfica *desktop*** (semestre seguinte) que monte o comando CLI e visualize os arquivos JSON de configuração.
- **Suporte a TSV/Excel** e a processamento *chunked* para CSV maiores que a RAM.
- **Operadores de filtro adicionais** e políticas de coerção de tipos configuráveis.
- **Testes de integração** contra docker compose em CI.
- **Generalização** do núcleo de importação para novos conectores (mensageria, *data lakes*, outros SGBDs).

6 CONSIDERAÇÕES FINAIS

Este TCC I apresentou a fundamentação teórica da persistência poliglota, o estado da arte em importação e modelagem de dados heterogêneos, e a proposta da ferramenta *Polyglot Import CSV* como utilitário de *bootstrap* declarativo a partir de CSV.

As principais contribuições desta etapa são: (i) um formato de configuração JSON validado estaticamente por JSON Schema, separado em mapeamento e conexão, com mapeamento recursivo de colunas e filtros por valor; (ii) um protótipo em Python que materializa o mesmo CSV operacional em PostgreSQL, MongoDB, Cassandra, Redis e Neo4j; (iii) o modo `--dry-run` para revisão prévia de contagens; e (iv) o cenário e-commerce reproduzível com `ecommerce_join.csv`, `import_config.json`, `sghd_config.json` e `docker-compose.yml`.

As limitações reconhecidas incluem: testes automatizados predominantemente unitários (sem suíte de integração contínua contra Docker); processamento em memória do CSV inteiro; e dependência do *driver* Cassandra em ambientes Python recentes. Itens como interface gráfica, suporte a TSV/Excel, *chunked processing* e novos conectores permanecem no escopo do TCC II (Capítulo 5).

O trabalho de conclusão de curso em dezembro de 2026 poderá aprofundar a avaliação experimental (desempenho, consistência entre *stores*) e a interface de uso, consolidando a ferramenta como artefato de pesquisa e de código aberto alinhado à literatura sobre SGPD e modelagem poliglota de dados.

REFERÊNCIAS

- BERLANGA, A. et al. A unified metamodel for nosql and relational databases. *arXiv preprint arXiv:2105.06494*, 2021. Citado na página 26.
- CHILLÓN, A. H. et al. A generic schema evolution approach for nosql and relational databases. *IEEE Transactions on Knowledge and Data Engineering*, v. 36, n. 7, p. 2774–2789, 2024. Citado na página 26.
- CHILLÓN, A. H. et al. Propagating schema changes to code: An approach based on a unified data model. In: . [S.l.: s.n.], 2023. Citado 2 vezes nas páginas 11 e 26.
- DBCROSSBAR: copy tabular data between databases, CSV files, and cloud storage. 2024. <<https://www.dbcrossbar.org>>. Ferramenta de código aberto em Rust para migração e conversão de esquemas. Citado 2 vezes nas páginas 25 e 26.
- DUGGAN, J. et al. Bigdawg: A polystore system for big data analytics. In: *IEEE International Conference on Data Engineering Workshops*. [S.l.: s.n.], 2017. Citado na página 26.
- FORD, N. *The Productive Programmer*. [S.l.]: O'Reilly Media, 2008. Citado na página 14.
- FOWLER, M. *PolyglotPersistence*. 2011. Disponível em: <<https://martinfowler.com/bliki/PolyglotPersistence.html>>. Citado na página 14.
- GLAKE, D. et al. *Towards Polyglot Data Stores: Overview and Open Research Questions*. 2022. Citado 3 vezes nas páginas 20, 26 e 27.
- Hackolade. *Polyglot Data Modeling*. 2024. <<https://hackolade.com/polyglot-data-modeling.html>>. Modelagem conceitual com engenharia de esquema para múltiplos SGBDs. Citado na página 26.
- HECHT, R.; JABLONSKI, S. Nosql evaluation: A use case oriented survey. In: *Proceedings of the 2011 International Conference on Cloud and Service Computing*. [S.l.: s.n.], 2011. p. 336–341. Citado na página 16.
- KHINE, P. P.; WANG, Z. A review of polyglot persistence in the big data world. *Information*, v. 10, n. 4, p. 141, 2019. Citado na página 11.
- KIEHN, F. et al. Polyglot data management: State of the art & open challenges. *Proceedings of the VLDB Endowment*, v. 15, n. 12, p. 3750–3753, 2022. Citado 4 vezes nas páginas 11, 20, 26 e 27.
- MongoDB, Inc. *mongoimport — Database Tools*. 2026. <<https://www.mongodb.com/docs/database-tools/mongoimport/>>. Utilitário de importação de CSV/JSON/TSV para o MongoDB. Citado na página 24.
- Neo4j, Inc. *Import — Operations Manual*. 2026. <<https://neo4j.com/docs/operations-manual/current/tools/neo4j-admin/neo4j-admin-import/>>. Importação em massa de CSV com neo4j-admin database import. Citado na página 25.

PEREIRA, F. d. O.; GUERRA, E.; ROSA, R. R. A framework model for supporting transparent polyglot persistence with a unified API and extensible for different database types. In: *Proceedings of the 27th International Conference on Enterprise Information Systems (ICEIS)*. [S.l.]: SciTePress, 2025. p. 109–120. Citado na página 26.

Redis Ltd. *Bulk Loading*. 2026. <<https://redis.io/docs/latest/develop/clients/patterns/bulk-loading/>>. Carga em massa via `redis-cli --pipe` usando o protocolo RESP. Citado na página 23.

ROY-HUBARA, N.; SHOVAL, P.; STURM, A. Selecting databases for polyglot persistence applications. *Data & Knowledge Engineering*, v. 137, p. 101950, 2022. Citado 2 vezes nas páginas 20 e 26.

SACHDEVA, S.; VASAVA, J. Plugging and playing with variety of data using multi-model database and polyglot persistence. In: *2024 Third International Conference on Electrical, Electronics, Information and Communication Technologies (ICEEICT)*. [S.l.]: IEEE, 2024. p. 1–8. Citado na página 26.

SADALAGE, P. J.; FOWLER, M. *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. [S.l.]: Pearson Education, 2013. Citado 2 vezes nas páginas 11 e 20.

SILVA, H. A. B. d.; BORNIA, L. G.; MELLO, R. d. S. Um estudo sobre modelagem poliglota de dados. In: *Anais da Escola Regional de Banco de Dados (ERBD)*. [S.l.]: Sociedade Brasileira de Computação, 2024. p. 11–20. Citado 2 vezes nas páginas 11 e 26.

TAN, R. et al. Enabling query processing across heterogeneous data models: A survey. In: *2017 IEEE International Conference on Big Data (BigData)*. [S.l.: s.n.], 2017. p. 3211–3220. Citado 2 vezes nas páginas 20 e 26.

The Apache Software Foundation. *sstableloader*. 2026. <<https://cassandra.apache.org/doc/latest/cassandra/managing/tools/sstable/sstableloader.html>>. Ferramenta de carga em massa de SSTables no Apache Cassandra. Citado na página 24.

The PostgreSQL Global Development Group. *PostgreSQL Documentation: COPY*. 2026. <<https://www.postgresql.org/docs/current/sql-copy.html>>. Comando COPY para importação/exportação de dados, inclusive CSV. Citado na página 22.

WANG, L. et al. A middleware for polyglot persistence and data portability of big data paas cloud applications. *Computers, Materials & Continua*, v. 65, n. 2, p. 1399–1416, 2020. Citado na página 26.

YE, F. et al. A benchmark for performance evaluation of a multi-model database vs. polyglot persistence. *Journal of Database Management*, v. 34, n. 3, p. 1–20, 2023. Citado 2 vezes nas páginas 11 e 14.

APÊNDICES

APÊNDICE A – JSON SCHEMAS DE CONFIGURAÇÃO NA FORMA SERIALIZADA

Este apêndice reproduz os dois documentos *JSON Schema* (rascunho 2020-12) tal como embutidos em `src/polyglotimportcsv/schemas/`. A estrutura de alto nível de ambos é descrita na seção 4.2; os fragmentos por SGBD são apresentados nas respectivas subseções do capítulo 4.

A.1 SCHEMA DE IMPORTAÇÃO (IMPORT_CONFIG.SCHEMA.JSON)

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://polyglot-import-csv.local/schemas/import_config.schema.json",
  "title": "PolyglotImportCSV import configuration",
  "description": "Declarative mapping from a wide CSV file to entities and
    relationships of one or more database backends. Connection settings live in
    the separate SGBD configuration.",
  "type": "object",
  "required": ["version"],
  "properties": {
    "version": {
      "type": "integer",
      "minimum": 1,
      "description": "Configuration format version."
    },
    "postgres": { "$ref": "#/$defs/postgresMapping" },
    "mongodb": { "$ref": "#/$defs/mongoMapping" },
    "cassandra": { "$ref": "#/$defs/cassandraMapping" },
    "redis": { "$ref": "#/$defs/redisMapping" },
    "neo4j": { "$ref": "#/$defs/neo4jMapping" }
  },
  "additionalProperties": false,
  "$defs": {
    "columnSpec": {
      "type": "object",
      "description": "Leaf mapping from a CSV column to a destination field.",
      "properties": {
        "is_key": {
          "type": "boolean",
          "description": "Primary or merge key for the entity."
        }
      },
    },
    "csv_column": {
      "description": "CSV header name or 0-based column index.",
      "oneOf": [
        { "type": "string", "minLength": 1 },
        { "type": "integer", "minimum": 0 }
      ]
    },
    "schema_column": {
      "type": "string",
      "minLength": 1,
    }
  }
}
```

```

        "description": "Destination field name when it differs from the JSON key."
    },
    "db_type": {
        "type": "string",
        "description": "SQL/CQL type for DDL generation (PostgreSQL, Cassandra)."
    }
},
"additionalProperties": false
},
"columnEntry": {
    "description": "Either a leaf columnSpec or a nested object (MongoDB
        subdocuments).",
    "oneOf": [
        { "$ref": "#/$defs/columnSpec" },
        {
            "type": "object",
            "minProperties": 1,
            "additionalProperties": { "$ref": "#/$defs/columnEntry" }
        }
    ]
},
"filter": {
    "type": "object",
    "required": ["column", "operator"],
    "properties": {
        "column": { "type": "string", "minLength": 1 },
        "operator": {
            "type": "string",
            "enum": ["=", "!=", ">", "<", ">=", "<=", "in", "not_in", "each"]
        },
        "value": {},
        "target_suffix": { "type": "string" }
    },
    "additionalProperties": false
},
"entity": {
    "type": "object",
    "required": ["columns"],
    "properties": {
        "columns": {
            "type": "object",
            "minProperties": 1,
            "additionalProperties": { "$ref": "#/$defs/columnEntry" }
        },
        "filters": {
            "type": "array",
            "items": { "$ref": "#/$defs/filter" }
        },
        "cassandra_partition": {
            "type": "array",
            "items": { "type": "string" }
        },
        "cassandra_cluster": {
            "type": "array",
            "items": { "type": "string" }
        }
    }
}

```

```
    },
    "additionalProperties": false
  },
  "postgresMapping": {
    "type": "object",
    "required": ["entities"],
    "properties": {
      "entities": {
        "type": "object",
        "additionalProperties": { "$ref": "#/$defs/entity" }
      },
      "relationships": {
        "type": "object",
        "additionalProperties": {
          "type": "object",
          "required": ["from", "to", "foreign_key"],
          "properties": {
            "from": { "type": "string" },
            "to": { "type": "string" },
            "foreign_key": { "type": "string" },
            "references_key": { "type": "string" }
          }
        },
        "additionalProperties": false
      }
    },
    "additionalProperties": false
  },
  "mongoMapping": {
    "type": "object",
    "required": ["entities"],
    "properties": {
      "entities": {
        "type": "object",
        "additionalProperties": { "$ref": "#/$defs/entity" }
      }
    },
    "additionalProperties": false
  },
  "cassandraMapping": {
    "type": "object",
    "required": ["entities"],
    "properties": {
      "entities": {
        "type": "object",
        "additionalProperties": { "$ref": "#/$defs/entity" }
      }
    },
    "additionalProperties": false
  },
  "redisMapping": {
    "type": "object",
    "required": ["entities"],
    "properties": {
      "entities": {
        "type": "object",
```

```

        "additionalProperties": { "$ref": "#/$defs/entity" }
    },
    "additionalProperties": false
},
"neo4jMapping": {
    "type": "object",
    "required": ["entities"],
    "properties": {
        "entities": {
            "type": "object",
            "additionalProperties": { "$ref": "#/$defs/entity" }
        },
        "relationships": {
            "type": "object",
            "additionalProperties": {
                "type": "object",
                "required": ["from", "to", "type"],
                "properties": {
                    "from": { "type": "string" },
                    "to": { "type": "string" },
                    "type": { "type": "string" },
                    "columns": {
                        "type": "object",
                        "additionalProperties": { "$ref": "#/$defs/columnSpec" }
                    }
                }
            },
            "additionalProperties": false
        }
    },
    "additionalProperties": false
}
}
}

```

A.2 SCHEMA DE CONEXÃO (SGBD_CONFIG.SCHEMA.JSON)

```

{
    "$schema": "https://json-schema.org/draft/2020-12/schema",
    "$id": "https://polyglot-import-csv.local/schemas/sghd_config.schema.json",
    "title": "PolyglotImportCSV SGBD connection configuration",
    "description": "Connection settings for each database backend.",
    "type": "object",
    "required": ["version"],
    "properties": {
        "version": { "type": "integer", "minimum": 1 },
        "postgres": { "$ref": "#/$defs/postgresConnection" },
        "mongodb": { "$ref": "#/$defs/mongoConnection" },
        "cassandra": { "$ref": "#/$defs/cassandraConnection" },
        "redis": { "$ref": "#/$defs/redisConnection" },
        "neo4j": { "$ref": "#/$defs/neo4jConnection" }
    },
    "additionalProperties": false,
    "$defs": {

```

```
"postgresConnection": {
  "type": "object",
  "properties": {
    "connection": {
      "type": "object",
      "properties": {
        "host": { "type": "string" },
        "port": { "type": "integer" },
        "database": { "type": "string" },
        "user": { "type": "string" },
        "password": { "type": "string" }
      }
    },
    "additionalProperties": false
  },
  "schema": {
    "type": "string",
    "default": "public"
  }
},
"additionalProperties": false
},
"mongoConnection": {
  "type": "object",
  "properties": {
    "connection": {
      "type": "object",
      "properties": {
        "uri": { "type": "string" },
        "database": { "type": "string" }
      }
    },
    "required": ["uri", "database"],
    "additionalProperties": false
  }
},
"additionalProperties": false
},
"cassandraConnection": {
  "type": "object",
  "properties": {
    "connection": {
      "type": "object",
      "properties": {
        "hosts": { "type": "array", "items": { "type": "string" } },
        "port": { "type": "integer" },
        "keyspace": { "type": "string" },
        "protocol_version": { "type": "integer" }
      }
    },
    "required": ["hosts", "keyspace"],
    "additionalProperties": false
  }
},
"additionalProperties": false
},
"redisConnection": {
  "type": "object",
  "properties": {
```

```
    "connection": {
      "type": "object",
      "properties": {
        "host": { "type": "string" },
        "port": { "type": "integer" },
        "db": { "type": "integer" },
        "password": { "type": "string" }
      },
      "additionalProperties": false
    },
    "additionalProperties": false
  },
  "neo4jConnection": {
    "type": "object",
    "properties": {
      "connection": {
        "type": "object",
        "properties": {
          "uri": { "type": "string" },
          "user": { "type": "string" },
          "password": { "type": "string" },
          "database": { "type": "string" }
        },
        "required": ["uri", "user", "password"],
        "additionalProperties": false
      },
      "additionalProperties": false
    },
    "additionalProperties": false
  }
}
```